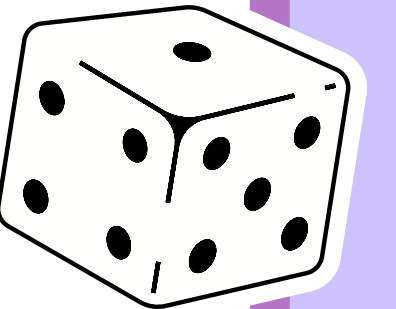
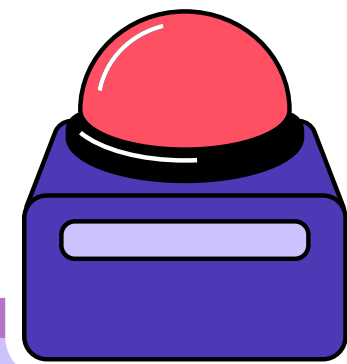


BCC1244 - GAME PROGRAMMING



SCENE MANAGEMENT



START



SCENE MANAGEMENT IN UNITY

Scene management in Unity refers to the organization, loading, and transitioning between different scenes in a game. A scene in Unity is essentially a level or a section of the game, such as a main menu, a gameplay level, or a cutscene. Effective scene management is crucial for creating a seamless and efficient gaming experience.

≡ MENU

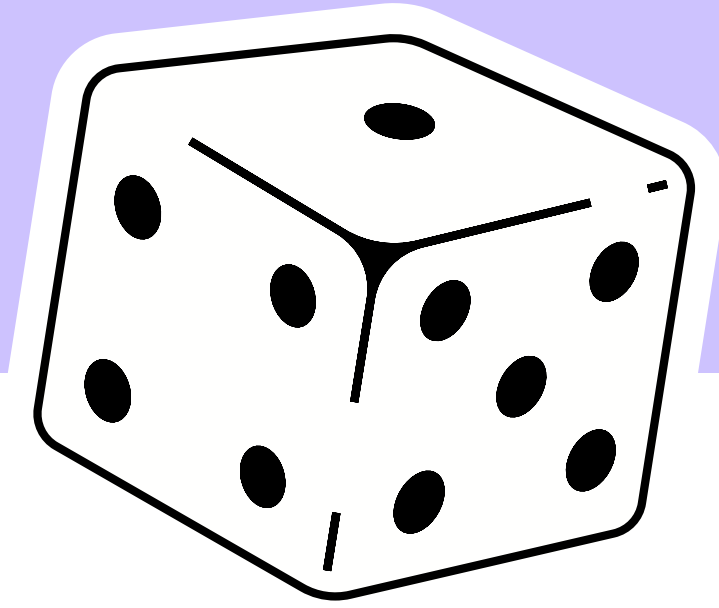
PLAY

SETTINGS

STORE

CREDITS

EXIT

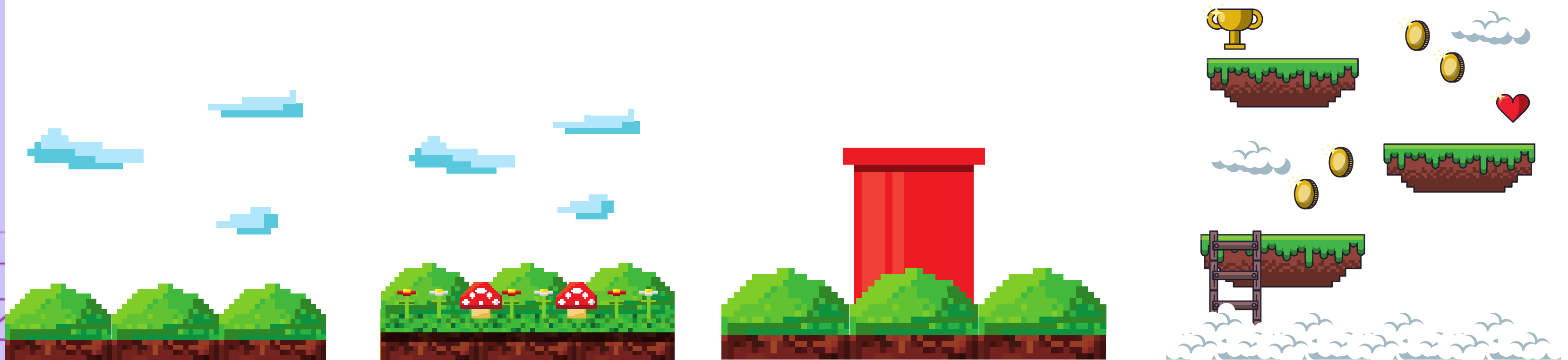


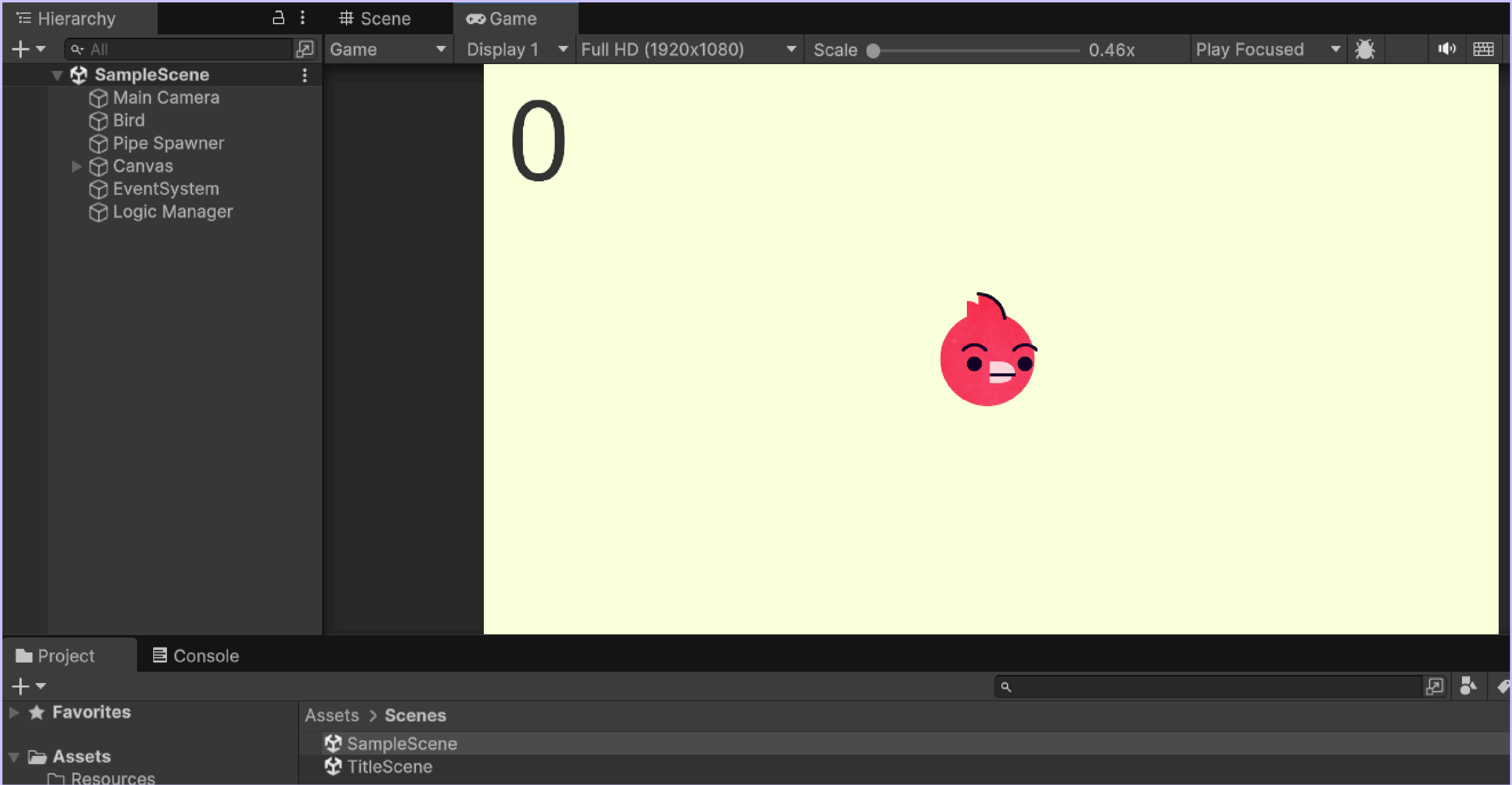
KEY ASPECTS OF SCENE MANAGEMENT

KEY ASPECTS

Scene Creation

Each scene is created and edited within the Unity Editor. You can add game objects, assets, scripts, and other elements to build the environment and logic for that specific part of the game.





KEY ASPECTS

Scene Loading

Unity provides the SceneManager class, which is part of the UnityEngine.SceneManagement namespace, to handle scene loading and unloading. You can load a scene additively (on top of the current scene) or singularly (replacing the current scene).

csharp

Copy

```
using UnityEngine.SceneManagement;

public void LoadScene(string sceneName)
{
    SceneManager.LoadScene(sceneName);
}
```

KEY ASPECTS

Scene Transition

Smooth transitions between scenes can be managed using various techniques such as fade-in/fade-out effects, loading screens, or animations. This helps maintain the immersion and flow of the game.



KEY ASPECTS

Persistent Data

Sometimes, you need to carry data between scenes, such as player scores, inventory, or game settings. This can be achieved using singleton patterns, static variables, or Unity's `DontDestroyOnLoad` method.

csharp

Copy

```
void Awake()
{
    DontDestroyOnLoad(gameObject);
}
```


KEY ASPECTS

Async Loading

To avoid freezing the game during scene transitions, Unity allows asynchronous scene loading. This means the game can continue running while the new scene is being loaded in the background.

csharp

Copy

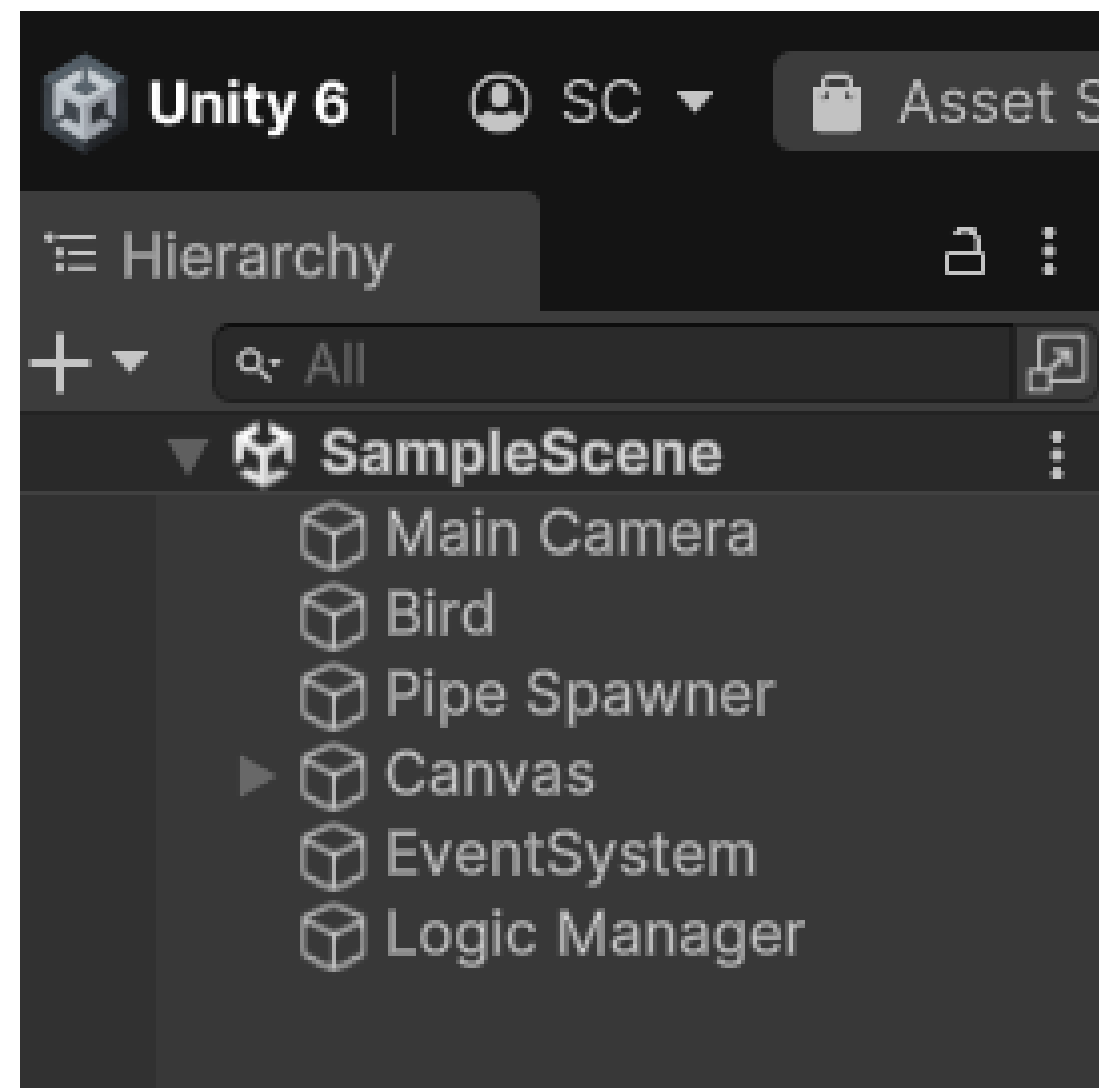
```
public IEnumerator LoadSceneAsync(string sceneName)
{
    AsyncOperation asyncLoad = SceneManager.LoadSceneAsync(sceneName);

    while (!asyncLoad.isDone)
    {
        yield return null;
    }
}
```

KEY ASPECTS

Scene Hierarchy and Organization

Properly organizing scenes within the Unity project is important for maintainability. This includes naming conventions, folder structures, and using scene dependencies effectively.



KEY ASPECTS

Scene Management Tools

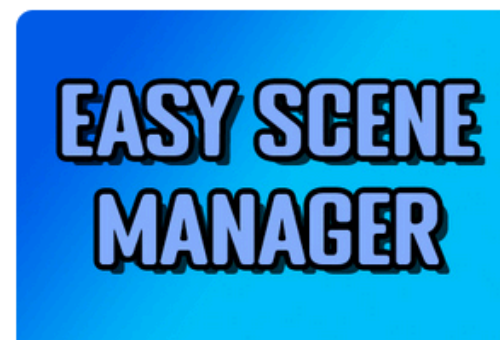
There are various tools and assets available in the Unity Asset Store that can help with scene management, such as scene transition managers, save/load systems, and more.



Advanced Scene Manager

★★★★★ 5 (32)
LAZY SOLUTIONS

\$68.00



Easy Scene Manager

(Not enough ratings)
HALF SLICE STUDIO

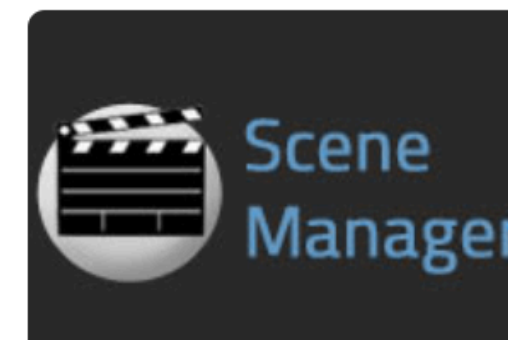
Free



Additive Scene Manager

★★★★☆ 4.7 (20)
RYAN ZEHR

\$25.00



Scene Manager

(Not enough ratings)
SIMPLE MINDED GAMES

Free



Scene Switch Manager

★★★★★ 5 (3)
SMASHLABS



Scene Manager Toolkit

(Not enough ratings)
LUDWELL STUDIO



SCEMA - scene manager
and loader

★★★★★ 4.8 (4)
RUNEMARK STUDIO



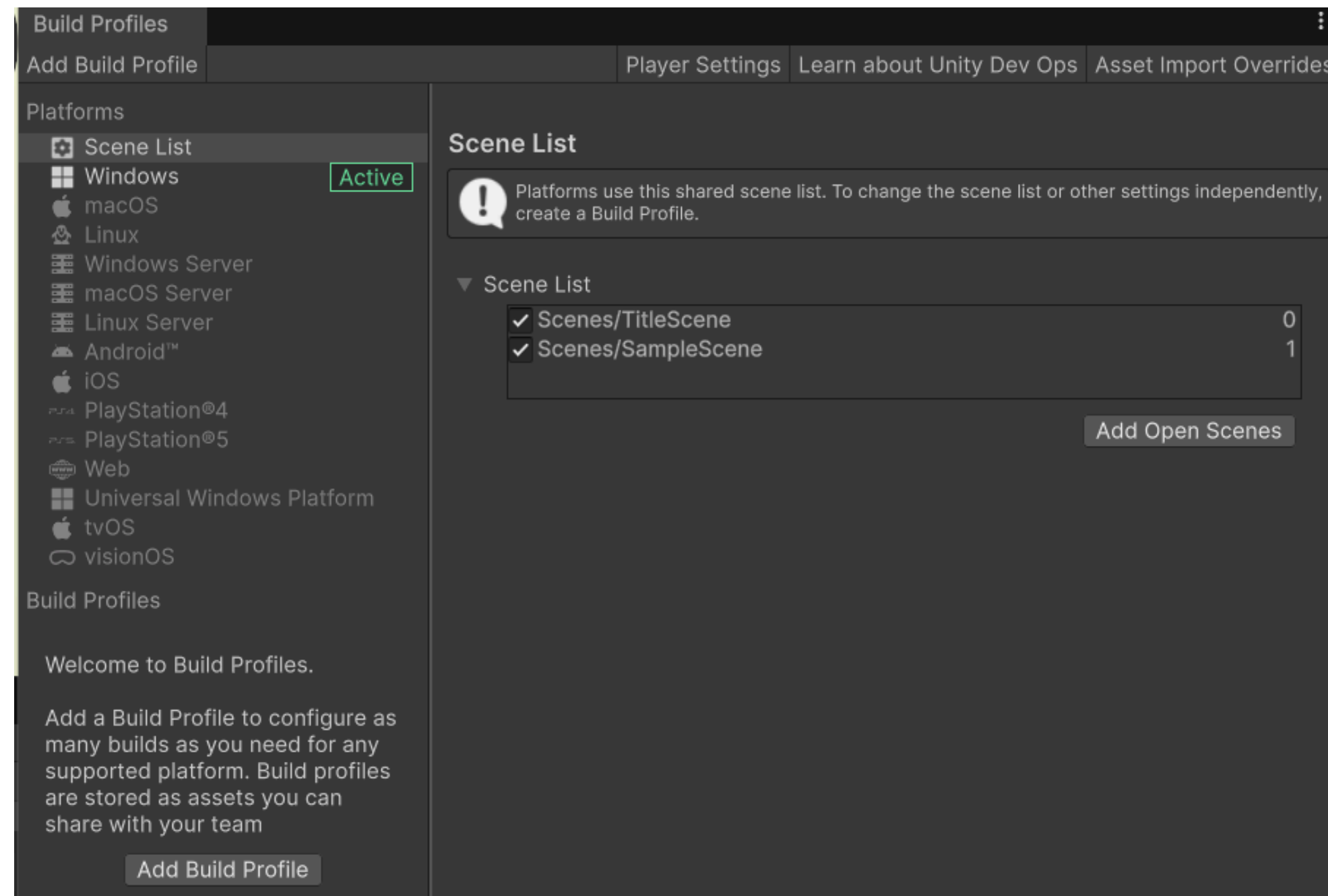
Visual Scene Manager

(Not enough ratings)
RAREBYTE OG

KEY ASPECTS

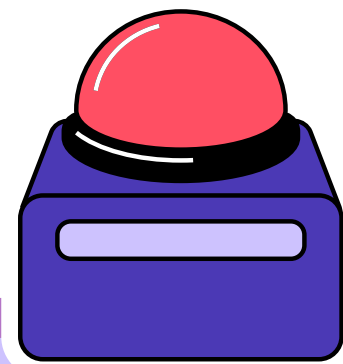
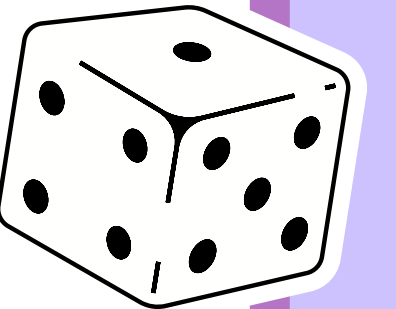
Build Settings

*Scenes need to be added to the build settings to be included in the final game build. This is done through the **File > Build Settings** or **File > Build Profile** menu, where you can drag and drop scenes into the build list.*



CONCLUSION

Effective scene management ensures that your game runs smoothly, loads efficiently, and provides a cohesive experience for the player. It involves a combination of good coding practices, thoughtful design, and leveraging Unity's built-in features and tools.





THANK YOU
FOR PLAYING!

